Java Backend Architecture

Interview Series

Chapter 18.6

Location Matching & Routing

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.6 – Location Matching & Routing

Overview

Nginx location blocks are the primary routing mechanism, directing requests to different backends, static files, or internal logic based on URI patterns. Understanding the priority order -- exact match, prefix no-regex, regex, prefix catch-all -- is essential to prevent routing mistakes in multi-service Java deployments. Named locations and try_files enable sophisticated routing without performance-degrading if statements.

Key Definitions

Term	Definition
= (exact)	Highest priority. Matches URI exactly. Processing stops immediately on match.
^~ (prefix no-regex)	Second priority. Prefix match that stops regex search. Faster than regex for static paths.
~ (case-sensitive regex)	Third priority. PCRE regex match. Evaluated in config file order.
~* (case-insensitive regex)	Same priority as ~. Evaluated in config file order.
/ (prefix catch-all)	Lowest priority. Matches everything. Used as default backend route.
Named location (@name)	Not directly accessible by clients. Used via try_files, error_page, internal redirects.
try_files	Checks file paths in order; serves first that exists; falls back to named location or URI.
rewrite	Transforms URI using PCRE. Flags: last (new location search), break (stop), redirect, permanent.
return	Sends HTTP response immediately (redirect or custom code). Faster than rewrite for simple cases.
internal	Restricts location to internal requests only (from try_files, error_page, rewrite).

Diagram 1: Location Matching Priority Order

```
Incoming request: GET /api/v1/users

Priority evaluation (stops at first match):
Step 1: Check exact matches (=)
        location = /health { ... }       -- NO match
        location = /favicon.ico { ... }  -- NO match

Step 2: Check prefix no-regex (^~)
        location ^~ /static/ { ... }     -- NO match

Step 3: Scan all regex locations (~, ~*)
        location ~ /api/v[0-9]+/ { ... } -- MATCH (stops here if found)
        location ~* .css$ { ... }        -- evaluated in order

Step 4: Longest prefix match from prefix locations
        location /api/ { ... }           -- if no regex matched
        location / { ... }               -- last resort
```

Diagram 2: try_files Fallback Chain

```
location / {
    try_files $uri $uri/ @app;
}

Request: GET /images/logo.png
Step 1: try $uri             -> check /var/www/html/images/logo.png (EXISTS -> serve)

Request: GET /dashboard
Step 1: try $uri             -> /var/www/html/dashboard (NOT EXISTS)
Step 2: try $uri/            -> /var/www/html/dashboard/ (index.html? NOT EXISTS)
Step 3: fallback @app        -> proxy_pass http://java_backend (React SPA or Spring MVC)
```

Code Examples

Full Routing Config for Java Microservices:

```
server {
    listen 443 ssl http2;
    server_name api.example.com;

    # Exact match -- fastest, no further search
    location = /health     { return 200 'OK'; access_log off; }
    location = /favicon.ico { return 204; access_log off; }

    # Prefix no-regex -- static assets, stops regex search
    location ^~ /static/ {
        root /var/www/html;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Case-insensitive regex
    location ~* \.(jpg|jpeg|png|gif|ico|css|js|woff2)$ {
        root /var/www/html;
        expires 30d;
    }

    # API routing by version
    location ~ ^/api/v2/ {
        proxy_pass http://java_v2_api;
    }

    # Catch-all -- default backend
    location / {
        try_files $uri $uri/ @java_app;
    }

    location @java_app {
        proxy_pass http://java_backend;
    }
}
```

Named Locations for Error Handling:

```
server {
    error_page 503 @maintenance;
    error_page 404 @not_found;

    location @maintenance {
        root /var/www/static;
        try_files /maintenance.html =503;
    }

    location @not_found {
        proxy_pass http://java_404_service;
    }

    location /api/ {
        proxy_pass http://java_api;
    }
}
```

Rewrite vs Return:

```
# return: immediate response (faster)
location /old-api/ {
    return 301 https://api.example.com/v2$request_uri;
}

# rewrite: transform URI, continue processing
location /app/ {
```

```
    rewrite ^/app/(.*)$ /$1 break;   # strip /app prefix, stop rewriting
    proxy_pass http://java_backend;
}

# Conditional routing via map (avoid if)
map $http_accept_language $lang_upstream {
    ~*^fr  french_api;
    ~*^de  german_api;
    default english_api;
}
location /api/ { proxy_pass http://$lang_upstream; }
```

Interview Q&A – Location Matching & Routing

Q1. What is the exact priority order of location matching?

1) Exact match (=): checked first, stops all processing on match. 2) Prefix no-regex (^~): longest matching prefix found, stops regex search. 3) Regex (~, ~*): evaluated in config file order, first match wins. 4) Longest prefix match: if no regex matched, use longest prefix. Tip: = for health checks eliminates regex scan. ^~ for /static/ prevents regex check.

Q2. Why is 'if is evil' inside location blocks?

if in location creates an implicit nested location with confusing inheritance. Only some directives work safely inside if (return, rewrite). Directives like proxy_pass, add_header behave unexpectedly. Example bug: if ($arg_debug) { add_header X-Debug 1; } causes add_header from parent location to be lost. Replace with map + $variable for conditional logic, try_files for existence checks.

Q3. How does try_files work for a React SPA behind Java backend?

try_files $uri $uri/ /index.html: if exact file exists, serve it (JS/CSS/images). If directory index exists, serve it. Otherwise serve index.html (React Router handles client-side routing). Java API routes must be defined before the SPA catch-all: location /api/ { proxy_pass http://java_backend; } must precede location / { try_files ... }

Q4. What is the difference between rewrite last, break, redirect, permanent?

last: stops current rewrite processing, starts new location search with modified URI. break: stops rewrite processing, continues in current location (no new location search). redirect: 302 temporary redirect (browser changes URL). permanent: 301 permanent redirect (browser and search engines cache). For URI transformation within a location: use break. For new location lookup: use last.

Q5. How do named locations differ from regular locations?

Named locations (location @name) are not accessible directly by clients -- only via try_files fallback, error_page directives, and internal redirects. They bypass the normal location priority system. Useful for error handling (error_page 503 @maintenance), fallback backends (try_files ... @app), and recursive internal redirect logic without regex overhead.

Q6. How does Nginx handle the longest prefix match?

Nginx collects ALL prefix locations matching the request URI, selects the LONGEST match. If that prefix has ^~, stops regex search and uses it. Otherwise, checks regex locations in config order. If a regex matches, use it. If no regex matches, use the longest prefix. Example: /api/v1/users matches both /api/ and /api/v1/ -- /api/v1/ (longer) wins.

Q7. How do you implement path-based routing for multiple Java microservices?

Use prefix locations for each service path: location /users/ { proxy_pass http://user_service/; } location /orders/ { proxy_pass http://order_service/; } location /payments/ { proxy_pass http://payment_service/; } location / { proxy_pass http://gateway_service; } Each location is independent; use ^~ to prevent regex overhead for static paths.

Q8. How does the map module replace complex if-location logic?

map $source_variable $target { pattern value; default value; } evaluated lazily, O(1) hash lookup. Examples: map $host $upstream (virtual host routing), map $request_method $no_cache (cache bypass for POST), map $cookie_experiment $variant (A/B routing). Cleaner than if-in-location: avoids inheritance issues, evaluates once per request, config is declarative and readable.

Q9. How does error_page work with proxy backends?

error_page 502 503 504 /error-page.html serves a static error file. error_page 503 @maintenance routes to a named location for dynamic maintenance page. Requires proxy_intercept_errors on to intercept upstream errors (without it, upstream 503 bypasses Nginx error_page). Custom error pages maintain professional UX when Java backend is down for deployment.

Q10. How do you restrict HTTP methods per location?

limit_except GET POST { deny all; } inside a location allows only GET and POST, denying PUT/DELETE/PATCH from external clients. For REST APIs: expose write endpoints only on internal network. location /admin/ { limit_except GET { allow 10.0.0.0/8; deny all; } proxy_pass http://admin_service; } Admin mutations restricted to internal Java services only.